Reporting Bugs

This is important information that I wanted to include in this segment itself, since many readers have asked about how to report bugs to the mailing list. Though there is no strict format, the community has framed a preferred format that makes it easy to scan through the contents. If you report a bug in this format, it will get your bug report more attention.

You may have noticed that many distributions have automatic reporting systems that allow you to report kernel-specific bugs to the maintainers. You can also do the same manually. First, you need to find the maintainer(s) of the kernel area that the bug is related to. For example, if the bug is related to the USB subsystem, then you would need to contact Greg Kroah-Hartman. You can find this out by issuing the following command:

```
perl scripts/get_maintainer.pl -f <filename>
```

When we discussed Git, we saw different ways to find details about the contributors. You could also CC the relevant mailing list(s). If you find it hard to trace the maintainer(s), then you can send your report to linux-kernel@vger.kernel.org. Here is the preferred format of the e-mail, as suggested by Frohwalt Egerer:

```
"PROBLEM: <one line summary from [1.]>" for easy identification by developers.

[1.] One-line summary of the problem:
[2.] Full description of the problem/report:
[3.] Keywords (i.e. modules, networking, kernel):
[4.] Kernel information
[4.1.] Kernel version (from /proc/version):
[4.2.] Kernel .config file:
[5.] Most recent kernel version which did not have the bug:2[6.] Output of Oops.. message (if applicable), with symbolic information resolved (see
Documentation/oops-tracing.txt)
[7.] A small shell script or example program which triggers the problem (if possible)
[8.] Environment
[8.1.] Software (add the output of the scripts/ver_linux script here)
[8.2.] Processor information (from /proc/cpuinfo):
[8.3.] Module information (from /proc/modules):
[8.4.] Loaded driver and hardware information (/proc/ioports, /proc/iomem)
[8.5.] PCI information ('lspci -vvv' as root)
[8.6.] SCSI information (from /proc/scsi/scsi)
[8.7.] Other information that might be relevant to the problem (please look in /proc and include all information that you think to be relevant):
[X.] Other notes, patches, fixes, workarounds:
```

Note: If you get an 'oops error', then you can supply more information to the maintainer than the displayed error message. Please consult 'Documentation/oops-tracing.txt' for more information.

```
        .slab_flags     = SLAB_DESTROY_BY_RCU,
        .twsk_prot      = &tcp6_timewait_sock_ops,
        .rsk_prot       = &tcp6_request_sock_ops,
        .h.hashinfo     = &tcp_hashinfo,
#ifdef CONFIG_COMPAT
        .compat_setsockopt   = compat_tcp_setsockopt,
        .compat_getsockopt   = compat_tcp_getsockopt,
#endif
};
```

Buffering

Another important aspect in networking is buffering. Here is code that performs a simple buffering task:

```
int npages;
        int i;

        /* No pages, we're done... */
        if (!data_len)
          break;

        npages = (data_len + (PAGE_SIZE - 1)) >> PAGE_SHIFT;
        skb->truesize += data_len;
        skb_shinfo(skb)->nr_frags = npages;
        for (i = 0; i < npages; i++) {
          struct page *page;
          skb_frag_t *frag;w

          page = alloc_pages(sk->sk_allocation, 0);
          if (!page) {
err = -ENOBUFS;
skb_shinfo(skb)->nr_frags = i;
kfree_skb(skb);
goto failure;
          }

          frag = &skb_shinfo(skb)->frags[i];
          frag->page = page;
          frag->page_offset = 0;
          frag->size = (data_len >= PAGE_SIZE ?
        PAGE_SIZE :
        data_len);
          data_len -= PAGE_SIZE;
        }
```

sk_wait_data() is another important function related to